09 - Excepciones

Pregunta número 1

In Java, which class is the superclass of all errors and exceptions?

Seleccione una:

- a. Object
- b. RuntimeException
- c. Exception
- d. Error
- e. Throwable

Respuesta: Lo primero que pensé es que, si dibujamos la jerarquía de excepciones de Java de arriba hacia abajo, en la cima, justo debajo de Object, encontramos a la clase Throwable. De ella se desprenden dos ramas: Error, que representa problemas graves de la JVM de los que normalmente no podemos recuperarnos (como OutOfMemoryError o StackOverflowError), y Exception, que agrupa las situaciones anómalas que sí podemos manejar en nuestro código. Solo los objetos que heredan de Throwable pueden usarse con throw o atraparse en un catch, por eso es la superclase de todos los errores y excepciones. El inciso a es tentador porque Object es la superclase de absolutamente todas las clases en Java, pero la pregunta se refiere específicamente a la jerarquía de errores y excepciones, cuya raíz es Throwable. Exception y Error son solo las dos ramas hijas, y RuntimeException está todavía un nivel más abajo, como subclase de Exception.

Pregunta número 2

¿Cuáles son los tipos de excepciones en Java?

Seleccione una o más de una:

- [ ] a. Throwable
- [ ] b. Object
- [x] c. Checked
- [x] d. Unchecked
- [ ] e. Todas las opciones son correctas.

Respuesta: Cuando hablamos de los tipos de excepciones en Java, la clasificación que importa es la que hace el compilador, y esa nos deja dos grupos. Por un lado, tenemos las excepciones checked (verificadas), que son todas las que heredan de Exception sin pasar por RuntimeException, como IOException o SQLException; el compilador nos obliga a manejarlas con un try-catch o a declararlas con throws. Por otro lado, están las unchecked (no verificadas), que son RuntimeException y sus subclases (junto con los Error), como NullPointerException o ArithmeticException; aquí el compilador no nos exige nada y el manejo queda a nuestra discreción. Throwable no es un tipo de excepción, sino la clase raíz de toda la jerarquía, y Object es simplemente la superclase de todas las clases, así que ninguno de los dos describe una categoría. Por esa misma razón, el inciso e tampoco puede ser correcto.

Pregunta número 3

¿Cuáles de las siguientes son palabras reservadas en Java para el manejo de excepciones?

Seleccione una:

- (●) a. try, catch, finally, throw, throws
- ( ) b. final, static, catch, throws
- ( ) c. throw & throws
- ( ) d. try, catch, final, throw, throws
- ( ) e. Error, Exception, RuntimeException, Throwable

Respuesta: Debemos recordar que en Java, el manejo de excepciones gira alrededor de cinco palabras reservadas, y el inciso a las reúne todas. Con try delimitamos el bloque de código que puede fallar; con catch atrapamos y manejamos la excepción; finally define un bloque que se ejecuta siempre, ocurra o no una excepción; throw sirve para lanzar una excepción de forma explícita, y throws se coloca en la firma de un método para declarar las excepciones que puede propagar. Yo descarto el inciso d por un detalle muy fácil de pasar por alto: dice final en lugar de finally. final es una palabra reservada, pero sirve para declarar constantes, métodos que no se pueden sobrescribir o clases que no se pueden heredar; no tiene nada que ver con las excepciones. El inciso b mezcla palabras ajenas al tema (final y static), el inciso c se queda corto porque solo menciona dos de las cinco, y el inciso e enumera nombres de clases de la jerarquía, no palabras reservadas.

Pregunta número 4

¿Para qué sirve la palabra clave throw en Java?

Seleccione una:

- a. Declarar excepciones en la firma de un método.
- b. Definir una clase personalizada de excepción.
- c. Capturar excepciones en un bloque try-catch.
- d. Crear y lanzar explícitamente una excepción.

Respuesta: Lo primero que debemos tener claro es que la palabra clave throw se usa dentro del cuerpo de un método para lanzar una excepción en un momento concreto, normalmente cuando detectamos una situación que no debería ocurrir. Va acompañada de una instancia de Throwable, que casi siempre creamos en ese mismo instante con new, por ejemplo, throw new IllegalArgumentException("Valor inválido"). En cuanto se ejecuta, el flujo normal se interrumpe y la excepción empieza a propagarse hasta que algún catch la atrape. Las demás opciones describen otros conceptos. Declarar excepciones en la firma de un método es el trabajo de throws, no de throw. Para definir una excepción personalizada, lo que hacemos es crear una clase que herede de Exception o de RuntimeException, y la captura se hace con el bloque catch. Por eso, la única descripción que corresponde a throw es la de crear y lanzar explícitamente una excepción.

Pregunta número 5

¿Cuál de las siguientes afirmaciones sobre la palabra clave throw es verdadera?

Seleccione una:

- a. Se utiliza para detectar excepciones generadas por otros métodos.
- b. Se utiliza para declarar que un método puede generar una excepción.
- c. Se utiliza para definir el código de limpieza que se debe ejecutar.
- d. Se utiliza para generar una excepción manualmente.

Respuesta: Esta pregunta posee prácticamente la misma idea que la anterior, pero redactada de otra forma, así que es un buen momento para fijar bien el concepto: throw es la instrucción con la que nosotros generamos una excepción de forma manual. No esperamos a que la JVM la produzca; somos nosotros quienes decidimos lanzarla al escribir algo como throw new Exception("mensaje").

Revisando los distractores, detectar excepciones generadas por otros métodos es la función del bloque catch, declarar que un método puede generar una excepción es la función de throws, y el código de limpieza que siempre debe ejecutarse va dentro del bloque finally. Me gusta pensarlo así: throw lanza, throws avisa, catch atrapa y finally limpia.

Pregunta número 6

In Java the difference between throws and throw Is:

Seleccione una:

- a. Throws throws an exception and throw indicates the type of exception that the method.
- b. Throws is used in methods and throw in constructors.
- c. Throws indicates the type of exception that the method does not handle and throw an exception.

Respuesta: En este ejercicio debemos tener presente la diferencia entre throw y throws, y justo el inciso c la resume correctamente. throws se escribe en la firma de un método y sirve para indicar qué tipos de excepciones puede propagar ese método sin manejarlas él mismo; es una forma de avisarle a quien lo llame que debe hacerse cargo de ellas. throw, en cambio, es la instrucción que efectivamente lanza una excepción en tiempo de ejecución. El inciso a invierte justamente los papeles, por lo que es incorrecto. El inciso b tampoco tiene sentido, ya que ambas palabras se pueden usar tanto en métodos como en constructores: un constructor puede declarar throws en su firma y también puede ejecutar un throw en su cuerpo.

Pregunta número 7

Which of the following statements correctly identifies checked and unchecked exceptions in Java?

Seleccione una:

- a. SQLException is an unchecked exception, and ArrayIndexOutOfBoundsException is a checked exception.
- b. IOException is a checked exception, and NullPointerException is an unchecked exception.
- c. RuntimeException is a checked exception, and FileNotFoundException is an unchecked exception.
- d. ArithmeticException is a checked exception, and ClassNotFoundException is an unchecked exception.

Respuesta: Para resolver esta pregunta hay una regla muy sencilla que podemos aplicar: si la excepción hereda de RuntimeException, es unchecked; si hereda de Exception sin pasar por RuntimeException, es checked. IOException hereda directamente de Exception, por lo que es checked y el compilador nos obliga a manejarla o declararla. NullPointerException hereda de RuntimeException, así que es unchecked. Por eso el inciso b es el único que clasifica bien ambas. En el inciso a, SQLException es checked y ArrayIndexOutOfBoundsException es unchecked, justo al revés de lo que dice. En el inciso c, RuntimeException es por definición la raíz de las unchecked, y FileNotFoundException es una subclase de IOException, así que es checked; de nuevo están invertidas. En el inciso d pasa lo mismo: ArithmeticException es unchecked y ClassNotFoundException es checked.

Pregunta número 8

What is the result?

```
public class Test {
    public static void main(String[] args) {
        try {
            mf();
            System.out.print("A");
        } catch (Exception e) {
            System.out.print("B");
        } finally {
            System.out.print("C");
        }
        System.out.print("D");
    }
    static void mf() {
    }
}
```

Seleccione una:

- a. ABCD
- b. ACD
- c. AC
- d. D
- e. compilation fails
- f. BCD

Respuesta: Para resolver este ejercicio, vamos a seguir el flujo del programa paso a paso. Dentro del try se llama a mf(), un método vacío que no hace nada y, por lo tanto, no lanza ninguna excepción. Como la ejecución continúa con normalidad, se imprime "A". Al no haber ocurrido ninguna excepción, el bloque catch se salta por completo, así que "B" nunca se imprime. Después viene el bloque finally, que se ejecuta siempre, haya o no excepción, e imprime "C". Finalmente, el programa sale de toda la estructura try-catch-finally y ejecuta la última instrucción, que imprime "D". El resultado es ACD. Tampoco hay ningún problema de compilación: atrapar Exception es válido aunque el try no lance nada, porque Exception incluye a las unchecked (RuntimeException). Esto no pasaría con una excepción checked específica. Los demás incisos se equivocan al incluir "B" o al omitir el finally o la última impresión.

Pregunta número 9

¿Qué ocurrirá si ejecutas el siguiente código?

```
public class TestThrow {
    public static void validarEdad(int edad) throws IllegalArgumentException {
        if (edad < 18) {
            throw new IllegalArgumentException("Edad no válida para votar.");
        }
        System.out.println("Edad válida.");
    }

    public static void main(String[] args) {
        try {
            validarEdad(16);
        } catch (IllegalArgumentException e) {
            System.out.println("Excepción capturada: " + e.getMessage());
        }
    }
}
```

Seleccione una:

- a. Se imprime "Edad válida."
- b. Lanza una excepción no controlada.
- c. Se imprime "Excepción capturada: Edad no válida para votar."
- d. No se imprime nada.

Respuesta: Lo primero de este ejercicio es que en el main llamamos a validarEdad(16). Como 16 es menor que 18, la condición del if es verdadera y se ejecuta throw new IllegalArgumentException("Edad no válida para votar."). En ese instante, el método se interrumpe, por lo que la línea que imprime "Edad válida." jamás se alcanza, y eso descarta el inciso a. La excepción se propaga hacia el

main, donde la llamada está dentro de un try cuyo catch atrapa justamente IllegalArgumentException. Ahí se ejecuta el println, que concatena el texto con e.getMessage(), y getMessage() regresa el mensaje que pasamos al constructor de la excepción. La salida es "Excepción capturada: Edad no válida para votar". El inciso b es incorrecto porque, aunque IllegalArgumentException es unchecked, aquí sí la estamos controlando con el catch, y el inciso d no aplica porque sí se imprime algo. Un detalle extra que noté es que el declarar throws IllegalArgumentException en la firma es opcional al ser unchecked, pero es válido y compila sin problema.

Pregunta número 10

Which three are bad practices?

Seleccione una o más de una:

- [ ] a. Checking for an IOException and ensuring that the program can recover if one occurs.
- [x] b. Checking for ArrayIndexOutOfBoundsException and ensuring that the program can recover if one occurs.
- [ ] c. Checking for FileNotFoundException to inform a user that a filename entered is not valid.
- [x] d. Checking for Error and, if necessary, restarting the program to ensure that users are unaware of problems.
- [x] e. Checking for ArrayIndexOutOfBoundsException when iterating through an array to determine when all elements have been visited.

Respuesta: Algo que hice en esta pregunta para identificar las malas prácticas es preguntarme en cada caso si estamos usando las excepciones para lo que fueron diseñadas, es decir, para manejar situaciones externas que no podemos controlar, y no para tapar errores de programación. El inciso b es una mala práctica, porque una ArrayIndexOutOfBoundsException indica un error en nuestra lógica (un índice mal calculado); lo correcto es corregir el código, no "recuperarnos" de ella. El inciso d también lo es: un Error representa un problema grave de la JVM, como quedarse sin memoria, que no deberíamos atrapar, y reiniciar el programa para ocultarles los problemas a los usuarios es esconder la falla en lugar de solucionarla. El inciso e es una mala práctica muy clara: usar la excepción para saber cuándo terminamos de recorrer un arreglo es usar excepciones como control de flujo, cuando para eso tenemos la propiedad length o un for-each. En cambio, los incisos a y c son buenas prácticas: una IOException o una FileNotFoundException dependen de factores externos, como un archivo que no existe o un disco que falla, y es completamente razonable manejarlas para recuperarnos o informar al usuario.

Pregunta número 11

¿Cuáles de las siguientes excepciones deben ser chequeadas?

Seleccione una o más de una:

- [x] a. IOException
- [ ] b. IndexOutOfBoundsException
- [x] c. InterruptedException
- [ ] d. NullPointerException
- [ ] e. ClassCastException
- [ ] f. ArithmeticException

Respuesta: Primero debemos recordar que una excepción checked es aquella que el compilador nos obliga a manejar con un try-catch o a declarar con throws, y en la jerarquía son todas las subclases de Exception que no heredan de RuntimeException. Con esa regla en mente, IOException es checked porque hereda directamente de Exception, y representa fallas de entrada y salida que no dependen de nuestro código. InterruptedException también es checked; la encontramos, por ejemplo, al usar Thread.sleep(), que nos obliga a manejarla porque otro hilo puede interrumpir al actual mientras espera. El resto de las opciones son unchecked, ya que todas heredan de RuntimeException: IndexOutOfBoundsException, NullPointerException, ClassCastException y ArithmeticException. Estas suelen indicar errores de programación; por eso el compilador no nos fuerza a manejarlas.

Pregunta número 12

Which of the following are unchecked exceptions?

Seleccione una o más de una:

- [x] a. ArrayIndexOutOfBoundsException
- [ ] b. IOException
- [x] c. RuntimeException
- [x] d. NumberFormatException
- [x] e. IllegalArgumentException
- [ ] f. InterruptedException

Respuesta: Esta pregunta es la contraparte de la anterior, porque ahora buscamos las unchecked. Recordemos que unchecked son RuntimeException y todas sus subclases. Sabemos que ArrayIndexOutOfBoundsException hereda de IndexOutOfBoundsException, que a su vez hereda de RuntimeException, así que es unchecked. RuntimeException, por ser la raíz de este grupo, también lo es. NumberFormatException es una subclase de IllegalArgumentException, y esta hereda de RuntimeException, por lo que ambas son unchecked; la primera la vemos cuando intentamos algo como Integer.parseInt("abc"). Las dos que quedan fuera son IOException e InterruptedException, que heredan directamente de Exception sin pasar por RuntimeException y, por lo tanto, son checked.

Pregunta número 13

¿Cuál será la salida del siguiente fragmento de código?

```
public class TestTryCatch {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
            System.out.println("Resultado: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Error: División por cero.");
        } catch (Exception e) {
            System.out.println("Error genérico.");
        } finally {
            System.out.println("Ejecución finalizada.");
        }
    }
}
```

Seleccione una o más de una:

- [x] a. Error: División por cero.
- [ ] b. Error genérico.
- [x] c. Ejecución finalizada.
- [ ] d. Resultado: 1

Respuesta: Lo primero que ocurre dentro del try es la división 10 / 0. Como estamos dividiendo enteros entre cero, la JVM lanza una ArithmeticException en ese

momento, por lo que la variable result nunca se asigna y la línea que imprimiría "Resultado:" no se ejecuta; con eso descartamos el inciso d. La excepción busca el primer catch compatible, revisándolos en orden de arriba hacia abajo. El primero es catch (ArithmeticException e), que coincide exactamente, así que se imprime "Error: División por cero". Solo se ejecuta un catch por excepción, de modo que el catch (Exception e) se ignora y "Error genérico" no aparece. Esto también nos confirma que el orden de los catch es correcto: el más específico va primero y el más general después; si estuvieran al revés, el código no compilaría. Finalmente, el bloque finally se ejecuta siempre e imprime "Ejecución finalizada.". Las dos líneas que aparecen en la salida son las de los incisos a y c.

Pregunta número 14

¿Cuál es la salida del siguiente código?

```
public class Test {
    public void run() {
        try {
            System.out.print("Running ");
            throw new Exception();
        } catch (Exception e) {
            System.err.print("Problem");
        } finally {
            System.out.print("Stopping");
        }
    }
    public static void main(String[] args) {
        new Test().run();
    }
}
```

Seleccione una:

- a. Running Stopping Problem
- b. Running Problem Stopping
- c. Running Problem
- d. La compilación falla
- e. Problem Stopping
- f. Stopping Problem

Respuesta: La clave de este ejercicio es que sigamos la ejecución del método run(). Dentro del try se imprime "Running " y en la siguiente línea se lanza una nueva Exception. Esta es checked, pero se atrapa en el mismo método con catch

(Exception e), así que el código compila sin problemas y el inciso d se descarta. El catch imprime "Problem" y, al terminar, se ejecuta el bloque finally, que imprime "Stopping". Por lo tanto, el orden de ejecución es Running, Problem, Stopping. Un detalle interesante que debemos notar es que "Problem" se imprime con System.err y no con System.out. Ambos flujos aparecen normalmente en la misma consola, aunque son independientes, pero el orden en que el código los ejecuta es el del inciso b. En la consola vemos "Running ProblemStopping", sin espacio antes de Stopping, porque ninguno de los textos lo agrega. Las demás opciones alteran el orden, colocando el finally antes del catch, u omiten alguna de las impresiones.

Pregunta número 15

What will be the output of the following code snippet?

```
import java.io.Closeable;
import java.io.IOException;

class Resource implements AutoCloseable {
    @Override
    public void close() {
        System.out.print("Resource closed ");
    }

    public void use() {
        System.out.print("Resource used ");
    }
}

public class Test {
    public static void main(String[] args) {
        try (Resource resource = new Resource()) {
            resource.use();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Seleccione una:

- a. Compilation error.
- b. Resource used Resource closed.
- c. Resource closed Resource used.
- d. Runtime error.

Respuesta: En este ejercicio, lo primero que debemos notar es que tenemos un try-with-resources, que funciona con cualquier objeto que implemente AutoCloseable. La clase Resource lo implementa y sobrescribe close() sin declarar ninguna excepción, lo cual es válido porque al sobrescribir un método podemos

lanzar menos excepciones que el original, nunca más. Los imports de Closeable e IOException no se usan, pero eso no provoca ningún error de compilación. Al ejecutarse, primero se crea el recurso en los paréntesis del try, después se ejecuta el cuerpo del bloque, que llama a use() e imprime "Resource used ", y en cuanto el bloque termina, Java llama automáticamente a close(), que imprime "Resource closed ". El cierre siempre ocurre al final del bloque try, y ocurre antes de que se ejecute cualquier catch o finally. Como no hubo ninguna excepción, el catch no se ejecuta. La salida es "Resource used Resource closed", lo que descarta el inciso c, que invierte el orden, y también los incisos a y d, porque el código compila y corre sin fallas.

Pregunta número 16

Which two possible outputs?

```
public class Main {
    public static void main(String[] args) throws Exception {
        doSomething();
    }
    private static void doSomething() throws Exception {
        System.out.println("Before if clause");
        if (Math.random() > 0.5) { throw new Exception();}
        System.out.println("After if clause");
    }
}
```

Seleccione una o más de una:

- [x] a. Before if clause Exception in thread "main" java.lang.Exception at Main.doSomething (Main.java:21) at Main.main (Main.java:15).
- [ ] b. Before if clause Exception in thread "main" java.lang.Exception at Main.doSomething (Main.java:21) at Main.main (Main.java:15) After if clause.
- [ ] c. Exception in thread "main" java.lang.Exception at Main.doSomething (Main.java:21) at Main.main (Main.java:15).
- [x] d. Before if clause After if clause.

Respuesta: La clave de este ejercicio es entender que el resultado de este programa depende de Math.random(), que regresa un número aleatorio entre 0.0 y 1.0, así que tenemos dos escenarios posibles. En ambos, lo primero que se ejecuta es el println de "Before if clause", por lo que cualquier salida válida tiene que empezar con ese texto, y eso descarta el inciso c. En el primer escenario, el número es mayor a 0.5 y se lanza una Exception. Como doSomething() la declara con throws y el main también, nadie la atrapa, así que llega a la JVM, que imprime el stack trace con el mensaje Exception in thread "main" y termina el programa. En ese caso, "After if clause" nunca se imprime, y eso corresponde al inciso a. En el

segundo escenario: el número es menor o igual a 0.5, no se lanza nada y el método continúa e imprime "After if clause", que es el inciso d. El inciso b es imposible, porque una vez que se lanza la excepción, el método se interrumpe y nunca podría imprimirse "After if clause" después del stack trace.

Pregunta número 17

Which two actions, used independently, will permit this class to compile?

```
import java.io.IOException;

public class Y {
    public static void main(String[] args) {
        try {
            doSomething();
        } catch (RuntimeException e) {
            System.out.println(e);
        }
    }

    static void doSomething() {
        if (Math.random() > 0.5) {
            throw new IOException();
        }
        throw new RuntimeException();
    }
}
```

Seleccione una o más de una:

- [x] a. Adding throws IOException to the main() method signature and to the doSomething() method
- [ ] b. Adding throws IOException to the doSomething() method signature
- [ ] c. Adding throws IOException to the main() method signature
- [x] d. Adding throws IOException to the doSomething() method signature and changing the catch argument to IOException
- [ ] e. Adding throws IOException to the main() method signature and changing the catch argument to IOException

Respuesta: El principal problema de este ejercicio es que doSomething() puede lanzar una IOException, que es checked, pero no la declara ni la maneja. Por la regla de "atrapar o declarar" (handle or declare), el método tiene que hacer una de las dos cosas, y si opta por declararla, la obligación pasa a quien lo llama. Revisemos cada opción. El inciso a funciona: doSomething() declara throws IOException y main() también la declara, así que la excepción se propaga sin problema; el catch de RuntimeException sigue siendo válido porque atrapa la otra

excepción. El inciso d también funciona: doSomething() la declara y en el main cambiamos el catch a IOException, con lo que el main sí la atrapa. Aquí la RuntimeException ya no se atrapa, pero como es unchecked, el compilador no exige nada. El inciso b falla porque, al declarar la excepción en doSomething(), el main recibe una IOException que no atrapa ni declara. El inciso c falla porque doSomething() sigue lanzando una IOException sin declararla. El inciso e tiene dos errores: doSomething() sigue sin declararla y, además, no podemos atrapar una excepción checked en un catch cuando el try no puede lanzarla.

Pregunta número 18

¿Cuáles son tres malas prácticas?

Seleccione una o más de una:

- [x] a. Comprobación de ArrayIndexOutOfBoundsException al iterar a través de una matriz para determinar cuándo se han visitado todos los elementos.
- [x] b. Verificar errores y, si es necesario, reiniciar el programa para garantizar que los usuarios no se den cuenta de los problemas.
- [ ] c. Verificar FileNotFoundException para informar a un usuario que un nombre de archivo ingresado no es válido.
- [x] d. Comprobar si hay ArrayIndexOutOfBoundsException y garantizar que el programa pueda recuperarse si ocurre alguna.
- [ ] e. Comprobar si hay una IOException y garantizar que el programa pueda recuperarse si ocurre alguna.

Respuesta: Esta es la misma pregunta que la 10, pero ahora en español; aunque los incisos cambiaron de orden, así que solo debemos tener cuidado al leer los incisos. El inciso a es una mala práctica, porque usar ArrayIndexOutOfBoundsException para saber cuándo terminamos de recorrer un arreglo es usar excepciones como control de flujo; para eso existe length o el for-each. El inciso b también lo es, ya que un Error representa una falla grave de la JVM que no deberíamos atrapar, y reiniciar el programa para que el usuario no se entere es esconder el problema en lugar de corregirlo. El inciso d es la tercera mala práctica: una ArrayIndexOutOfBoundsException indica un error de programación que debemos corregir en el código, no algo de lo que debamos recuperarnos. Los incisos c y e son buenas prácticas, porque FileNotFoundException e IOException dependen de factores externos (un archivo que no existe, una falla de lectura) y es correcto manejarlas para informar al usuario o recuperarnos.

Pregunta número 19

Que muestra en pantalla al ejecutar este código?

```
public class Test {
    int[] array = {1, 2, 3, 4, 5};
    public static void main(String[] args) {
        new Test().go(1);
    }
    void go(int x) {
        array[x % 5] = x++;
        go(x);
    }
}
```

Seleccione una:

- a. A java.lang.ArrayIndexOutOfBoundsException is thrown.
- b. A java.lang.IllegalStateException is thrown.
- c. A java.lang.StackOverflowError is thrown.
- d. The program runs with no output.
- e. Compilation fails

Respuesta: Lo primero que debemos dejar en claro es que el código compila sin problemas: array es un atributo de instancia, go() es un método de instancia y lo llamamos desde un objeto creado con new Test(). El problema está en la ejecución. go(1) asigna un valor a array[1 % 5], incrementa x a 2 y se llama a sí mismo con go(2); este a su vez llama a go(3), y así sucesivamente. Lo que podría preocuparnos es el índice, pero el operador % 5 garantiza que siempre estará entre 0 y 4, así que nunca nos salimos del arreglo y el inciso a queda descartado. El verdadero problema es que la recursión no tiene ningún caso base, es decir, ninguna condición que la detenga. Cada llamada agrega un nuevo marco a la pila (stack) de ejecución, y cuando la pila se llena, la JVM lanza un StackOverflowError. Debemos notar que es un Error, no una Exception. El inciso b no tiene relación con este código, y el inciso d es imposible porque el programa nunca termina de forma normal.

Pregunta número 20

Which three implementations are valid?

```
interface SampleCloseable {
    public void close() throws java.io.IOException;
}
```

Seleccione una o más de una:

- [x] a. class Test implements SampleCloseable { public void close() throws java.io.IOException { // do something } }
- [ ] b. class Test implements SampleCloseable { public void close() throws Exception { // do something } }
- [x] c. class Test implements SampleCloseable { public void close() throws FileNotFoundException { // do something } }
- [ ] d. class Test extends SampleCloseable { public void close() throws java.io.IOException { // do something } }
- [x] e. class Test implements SampleCloseable { public void close() { // do something }}

Respuesta: La clave de esta pregunta es que debemos aplicar las reglas de la sobrescritura de métodos con excepciones. Cuando una clase implementa el método close() de la interfaz, puede declarar la misma excepción, una más específica (una subclase) o ninguna, pero nunca puede declarar una excepción checked más general o nueva. El inciso a es válido porque declara exactamente la misma excepción, java.io.IOException. El inciso c también es válido, ya que FileNotFoundException es una subclase de IOException, es decir, una excepción más específica. El inciso e es válido porque no declarar ninguna excepción siempre está permitido, ya que nunca va a lanzar más de lo que promete la interfaz. El inciso b es inválido porque Exception es más general que IOException, y eso rompería el contrato de la interfaz. Finalmente, el inciso d falla por sintaxis: una clase no puede usar extends con una interfaz; debe usar implements.

Pregunta número 21

What is the result?

```
class Exc1 extends Exception {}
class Exc2 extends Exception {}

public class Test {
    static void a() throws Exc1 {
        try {
            System.out.print("Try ");
        } catch (Exc1 e) {
            System.out.print("Catch ");
        } finally {
            System.out.println("Finally ");
        }
    }

    static void b() throws Exc1, Exc2 {
        throw new Exc2();
    }

    public static void main(String args[]) {
        try {
            a();
            b();
        } catch (Exc1 e) {
            System.out.println("Caught1");
        } catch (Exc2 ex) {
            System.out.print("Caught2");
        }
    }
}
```

Seleccione una:

- a. Try Finally Caught2
- b. La compilación falla
- c. Try
- d. Try Caught2
- e. Try finally

Respuesta: En este ejercicio, lo primero que se me ocurrió es ponerme a calcular la salida, ya que pienso que podría llegar a "Try Finally Caught2", pero antes de eso creo que es importante revisar si el código compila. El problema está en el método a(). Su bloque try solo contiene un System.out.print("Try "), que no puede lanzar una

Exc1, y Exc1 es una excepción checked porque hereda directamente de Exception. El compilador de Java no permite tener un catch para una excepción checked que el bloque try nunca podría lanzar, y marca el error "exception Exc1 is never thrown in body of corresponding try statement". No importa que el método declare throws Exc1 en su firma; la regla se aplica al try y su catch. Por lo tanto, la compilación falla. Esta regla solo aplica a las excepciones checked; si el catch fuera de Exception, de RuntimeException o de cualquier unchecked, el código sí compilaría. El resto del código estaría bien: b() declara correctamente Exc1 y Exc2, y el main atrapa ambas, así que si quitáramos ese catch de a(), la salida sería la del inciso a.

Pregunta número 22

What is the result if you try to compile Truthy.java and then run it with assertions enabled?

```
public class Truthy {
    public static void main(String[] args) {
        int x=7;
        assert (x==6)?"x==6":"x!=6";
    }
}
```

Seleccione una:

- a. Truthy.java does NOT compile.
- b. Truthy.java compiles and the output is x != 6
- c. Truthy.java compiles and an AssertionError is thrown with no additional output
- d. Truthy.java compiles and an AssertionError is thrown with x != 6 as additional output

Respuesta: Lo primero que debemos recordar es que la sintaxis de assert tiene dos formas: assert condición; o assert condición : mensaje. En ambas, la primera expresión obligatoriamente debe ser de tipo boolean, ya que es la condición que se está verificando, y la segunda, que va después de los dos puntos, es el mensaje que acompaña al AssertionError. En este código, se intentó escribir un mensaje, pero el compilador lee toda la línea (x==6)?"x==6":"x!=6" como una sola expresión, porque los dos puntos se interpretan como parte del operador ternario. Ese ternario regresa un String ("x==6" o "x!=6"), no un boolean, así que no puede usarse como condición del assert y el código no compila. Para que funcionara, se tendría que escribir algo como assert (x==6) : "x!=6";. En ese caso, con las aserciones activadas (-ea), se lanzaría un AssertionError con el mensaje "x!=6", que sería el inciso d. Como el código está escrito en la pregunta, ni siquiera llega a ejecutarse, por lo que activar las aserciones no cambia nada.